Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ITMO University

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

По дисциплине: Контейнеризация и оркестрация приложений

Тема работы: Docker Compose: оркестрация сервисов с Redis-кэшированием

Обучающийся: Тер-Степанян Эвелина Сергеевна

Номер ИСУ: 467697

Факультет: Факультет прикладной информатики

Группа: К3321

Направление подготовки: 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа: Программирование в инфокоммуникационных системах

Обучающийся _____ Тер-Степанян Э. С.

дата, подпись

Руководитель _____ Аминов Н. С.

дата, подпись

Санкт-Петербург

2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**

ПЕРСОНАЛЬНЫЕ ПАРАМЕТРЫ **4**

1 Основная часть **5**

1.1 Этап 1. Подготовка проекта для Docker Compose 5

1.2 Этап 2. Добавление Redis для кэширования погоды 7

1.3 Этап 3. Запуск и проверка полного стека 10

1.4 Этап 4. Масштабирование сервиса 12

1.5 Этап 5. Финальный артефакт 14

ЗАКЛЮЧЕНИЕ **15**

ПРИЛОЖЕНИЕ А Содержимое docker-compose.yml **16**

ПРИЛОЖЕНИЕ Б Содержимое .env.example **18**

ПРИЛОЖЕНИЕ В Содержимое docker-compose.scale.yml . **19**

ВВЕДЕНИЕ

Цель работы — перенести запуск приложения, PostgreSQL и Redis в Docker Compose, добавить кэширование ответов приложения в Redis и попробовать масштабирование сервиса.

Задачи:

- вынести персональные параметры в файл `.env` и описать стек в `docker-compose.yml`;
- настроить порядок запуска сервисов через `healthcheck` и `depends_on`;
- добавить сервис Redis и кэширование эндпоинта `/weather/{city}`;
- проверить работу кэша, сетей и томов;
- запустить несколько экземпляров приложения и убедиться, что они используют общий кэш.

Работа выполнялась на сервере linux-lab, Docker версии 29.1.3 и Docker Compose версии 2.40.3 [?, ?].

ПЕРСОНАЛЬНЫЕ ПАРАМЕТРЫ

Персональные параметры приведены в таблице 1.

Таблица 1 — Персональные параметры

Параметр	Значение
Номер ИСУ	467697
Персональный порт (PostgreSQL, `PG_PORT`)	5527
Персональный порт (Redis, `REDIS_PORT`)	6467
Персональный порт (приложение, `APP_PORT`)	8097
Персональный префикс	ter-stepanyan
Секретный ключ	secret97ter-stepanyan
Имя базы данных	ter-stepanyan97

Репозиторий с результатом (ветка `lab-4`): https://github.com/LilllDrake/KUBER/tree/lab-4/solutions/assist-py.

1 Основная часть

1.1 Этап 1. Подготовка проекта для Docker Compose

От ветки `lab-3` создана ветка `lab-4`. В корне проекта `solutions/assist-py` создан файл `.env` с персональными переменными (рисунок 1). Файл содержит пароли, поэтому он добавлен в `.gitignore`, а в репозиторий помещён `.env.example` с теми же переменными без реальных паролей (приложение Б).

```
APP_OWNER=ter-stepanyan
APP_SECRET=secret97ter-stepanyan
APP_PORT=8097
POSTGRES_USER=user_97
POSTGRES_PASSWORD=pgpass_97
POSTGRES_DB=assistant_97
PG_PORT=5527
REDIS_PORT=6467
CACHE_TTL=120
```

Рисунок 1 — Заполнение файла .env

Создан файл `docker-compose.yml` с двумя сервисами: `db` (PostgreSQL) и `app` (приложение). Полное содержимое итогового файла приведено в приложении А. Основные решения:

- PostgreSQL запускается командой `postgres -p ${PG_PORT}`, поэтому внутри контейнера он слушает персональный порт 5527;
- данные базы хранятся в именованном томе `ter-stepanyan-pgdata`;
- у `db` задан `healthcheck` с командой `pg_isready`, которая проверяет, что база готова принимать подключения на порту 5527;
- приложение собирается из `Dockerfile.optimized`, слушает порт 8097, и этот порт пробрасывается на хост;

– приложение подключается к базе по строке `DATABASE_URL`, в которой хостом указано имя сервиса `db`, а не `localhost`: у каждого контейнера свой сетевой стек, и `localhost` внутри контейнера указывает на сам контейнер;

– папка `config` подключена как bind mount в режиме только для чтения, как в ЛР №3;

– приложение запускается только после того, как база пройдёт проверку здоровья (`depends_on` с `condition: service_healthy`).

Запуск всех сервисов командой `docker compose up -d --build` показан на рисунке 2. При первом запуске Compose добавил к именам сети и тома префикс проекта `assist-py_` (имя папки проекта). Чтобы сеть и том назывались ровно `ter-stepanyan-backend` и `ter-stepanyan-pgdata`, в описание сети и тома позже добавлен ключ `name`.

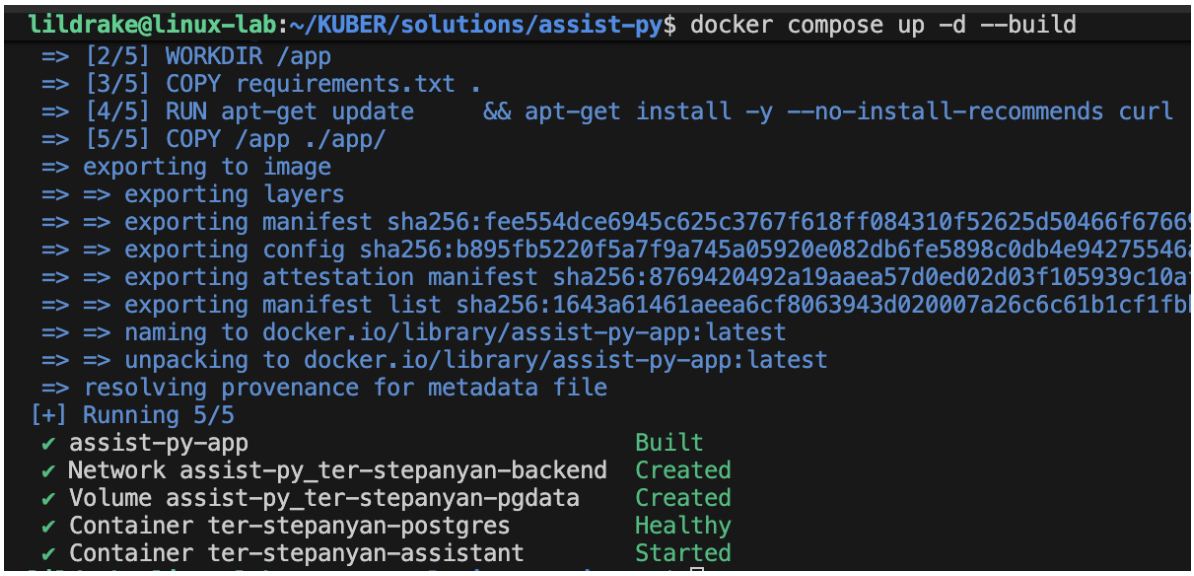


Рисунок 2 — Запуск сервисов через Docker Compose

Статус сервисов и логи приложения представлены на рисунке 3. Оба контейнера в состоянии `healthy`, приложение доступно на персональном порту 8097.



Рисунок 3 — Статус сервисов и логи приложения

В логах базы данных (рисунок 4) видно, что PostgreSQL слушает персональный порт 5527.

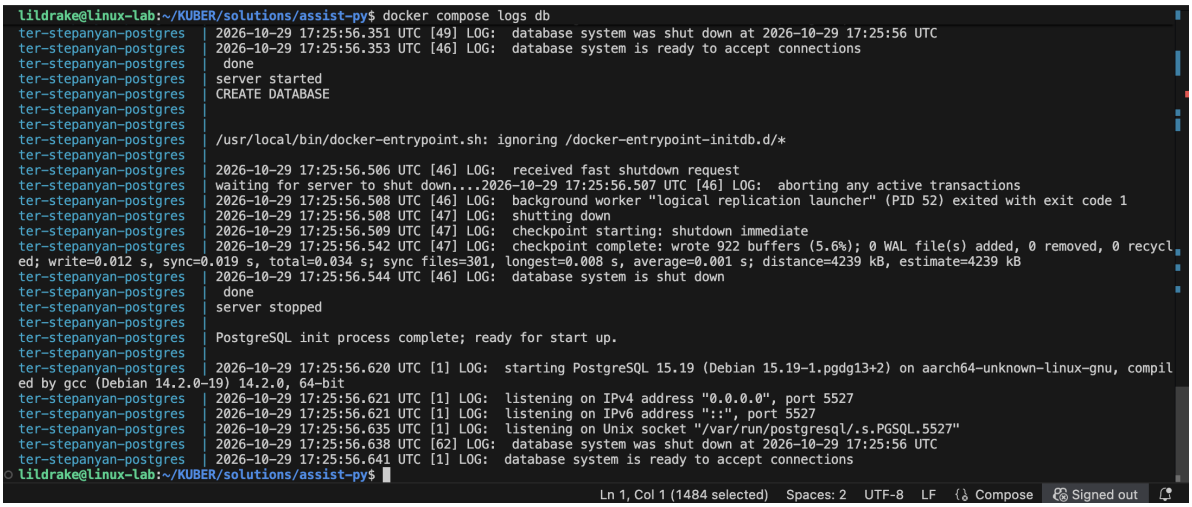


Рисунок 4 — Логи базы данных

Приложение отвечает на запросы `/` и `/health` (рисунок 5), в ответе указан владелец `ter-stepanyan`.



Рисунок 5 — Проверка ответа приложения

1.2 Этап 2. Добавление Redis для кэширования погоды

В `docker-compose.yml` добавлен сервис `redis` (рисунок 6) из образа `redis:7-alpine`. Он подключён к той же сети `ter-stepanyan-backend`, а его порт опубликован на хосте под персональным номером 6467. Проверка здоровья выполняется командой `redis-cli ping`.

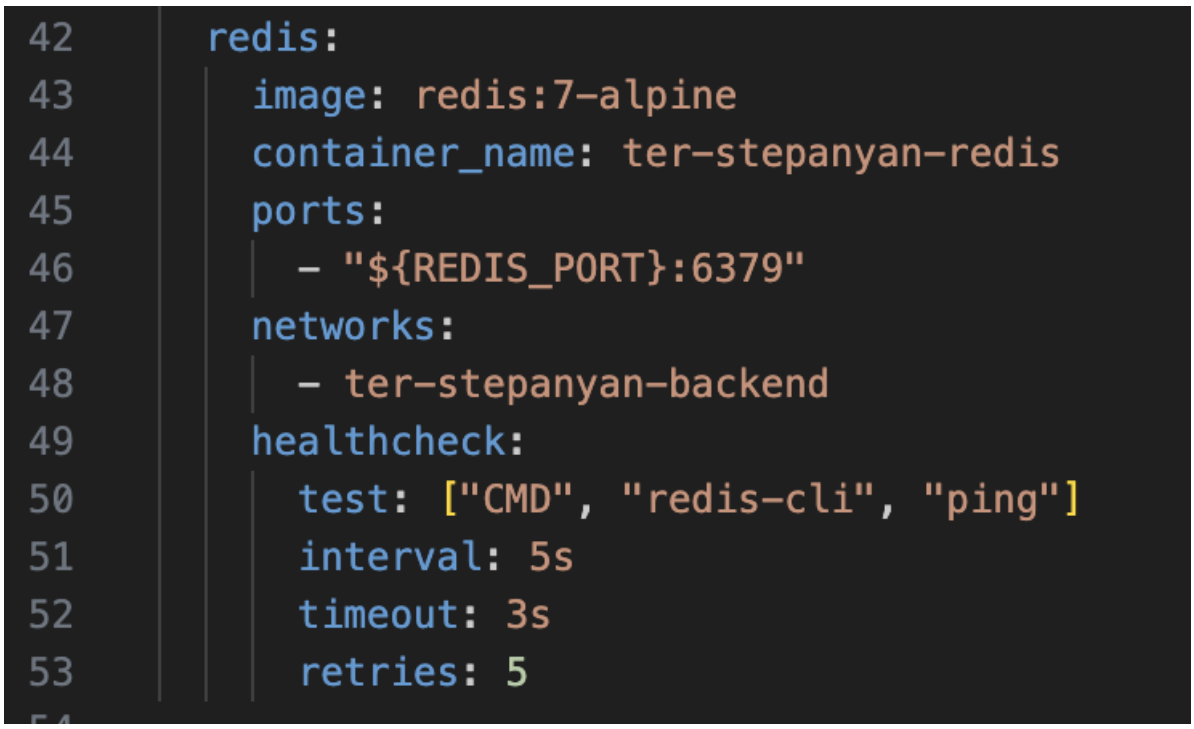


Рисунок 6 — Сервис redis в docker-compose.yml

Приложение теперь зависит и от базы, и от Redis (рисунок 7): Compose запустит его только после того, как оба сервиса станут `healthy`.

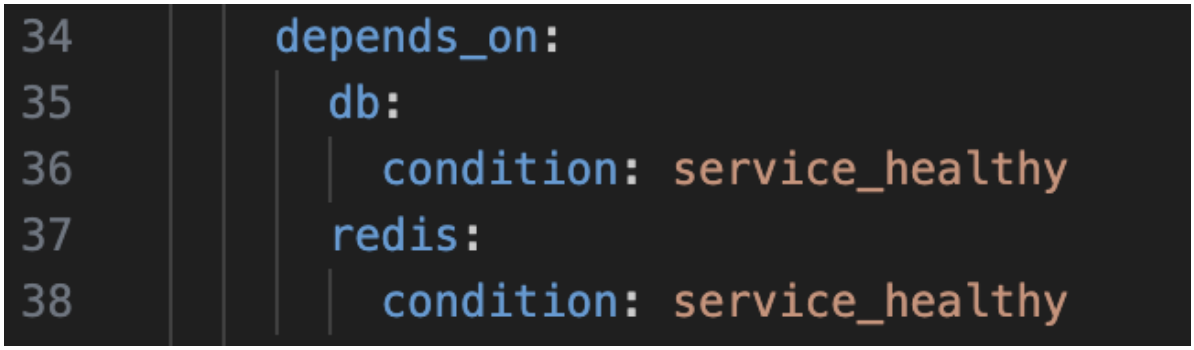


Рисунок 7 — Зависимость приложения от db и redis

В `app/main.py` добавлено подключение к Redis (рисунок 8), а в `requirements.txt` — библиотека `redis`. Адрес Redis и время жизни кэша приложение берёт из переменных окружения `REDIS_URL` и `CACHE_TTL`.

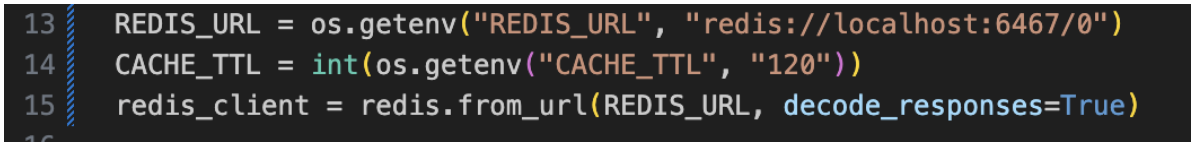


Рисунок 8 — Подключение к Redis в main.py

Эндпоинт `/weather/{city}` работает так:

1. приложение формирует ключ вида `weather:ter-stepanyan:moscow` и запрашивает его в Redis;
2. если значение найдено, ответ возвращается из кэша с полем `"source": "cache"` без обращения к внешнему API;
3. если значения нет, приложение запрашивает погоду у API open-meteo, возвращает ответ с `"source": "api"` и сохраняет его в Redis командой `SETEX` на `CACHE_TTL` = 120 секунд.

Так повторные запросы по одному городу не нагружают внешний API и обрабатываются быстрее, а через 120 секунд Redis сам удаляет ключ, и данные обновляются.

Также добавлен эндпоинт `/cache/stats` (рисунок 9). Он находит в Redis все ключи погоды текущего владельца и возвращает их количество и список городов.

```
139  @app.get("/cache/stats")
140  async def cache_stats():
141      keys = redis_client.keys(f"weather:{OWNER}:*")
142      return {
143          "owner": OWNER,
144          "ttl_seconds": CACHE_TTL,
145          "cached_count": len(keys),
146          "cached_cities": sorted(k.split(":")[-1] for k in keys),
147      }
148
```

Рисунок 9 — Эндпоинт статистики кэша

В переменные окружения приложения добавлены `REDIS_URL` и `CACHE_TTL` (рисунок 10). В `REDIS_URL` хостом указано имя сервиса `redis`, а портом — порт, на котором Redis слушает внутри своего контейнера. Проброс порта 6467 на хост для связи приложения с Redis не нужен: контейнеры общаются напрямую по пользовательской сети, а встроенный DNS Docker сопоставляет имя сервиса с IP-адресом контейнера. Проброшенный порт нужен только для доступа к Redis с хоста.

```
28      environment:
29        APP_OWNER: ${APP_OWNER}
30        APP_SECRET: ${APP_SECRET}
31        DATABASE_URL: postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:${PG_PORT}/${POSTGRES_DB}
32        REDIS_URL: redis://redis:6379/0
33        CACHE_TTL: ${CACHE_TTL}
```

Рисунок 10 — Переменные окружения приложения

1.3 Этап 3. Запуск и проверка полного стека

После пересборки все три сервиса запущены и находятся в состоянии `healthy` (рисунок 11): `ter-stepanyan-postgres`, `ter-stepanyan-assistant` и `ter-stepanyan-redis`. По выводу Compose видно, что `postgres` и `redis` стали `Healthy` через 5,6 с, а приложение запущено после них, через 5,8 с.

```
 ✔ assist-py-app                  Built                                                                                0.0s
 ✔ Container ter-stepanyan-assistant  Started                                                                          5.8s
 ✔ Container ter-stepanyan-postgres   Healthy                                                                          5.6s
 ✔ Container ter-stepanyan-redis      Healthy                                                                          5.6s
lildrake@linux-lab:~/KUBER/solutions/assist-py$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED              STATUS                    PORTS                                       NAMES
1615c3587f75   redis:7-alpine   "docker-entrypoint.s…"   Less than a second ago   Up 20 seconds (healthy)   0.0.0.0:6467->6379/tcp, [::]:6467->6379/tcp   ter-stepanyan-redis
7e162cb7a5d7   postgres:15      "docker-entrypoint.s…"   Less than a second ago   Up 20 seconds (healthy)   5432/tcp                                    ter-stepanyan-postgres
eb62a0b73b7b   assist-py-app    "uvicorn app.main:ap…"   21 seconds ago           Up 14 seconds (healthy)   0.0.0.0:8097->8097/tcp, [::]:8097->8097/tcp   ter-stepanyan-assistant
```

Рисунок 11 — Запуск полного стека

Логи всех сервисов представлены на рисунке 12. PostgreSQL и Redis сообщают о готовности принимать подключения (`database system is ready to accept connections` и `Ready to accept connections`), после чего стартует приложение. Предупреждение Redis `Memory overcommit must be enabled` относится к настройке ядра хоста и не влияет на работу кэша.

```
lildrake@linux-lab:~/KUBER/solutions/assist-py$ docker compose logs --tail=30
ter-stepanyan-postgres  | 2026-11-04 15:05:18.747 UTC [26] LOG:  checkpoint starting: end-of-recovery immediate wait
ter-stepanyan-postgres  | 2026-11-04 15:05:18.751 UTC [26] LOG:  checkpoint complete: wrote 3 buffers (0.0%); 0 WAL file(s) added, 0 removed, 0 recycled; write=0.001 s, sync=0.001 s, total=0.005
s; sync files=2, longest=0.001 s, average=0.001 s; distance=0 kB, estimate=0 kB
ter-stepanyan-postgres  | 2026-11-04 15:05:18.752 UTC [1] LOG:  database system is ready to accept connections
ter-stepanyan-redis     | 1:C 04 Nov 2026 15:11:39.699 # WARNING Memory overcommit must be enabled! Without it, a background save or replication may fail under low memory condition. Being disable
d, it can also cause failures without low memory condition, see https://github.com/jemalloc/jemalloc/issues/1328. To fix this issue add 'vm.overcommit_memory = 1' to /etc/sysctl.conf and then reb
oot or run the command 'sysctl vm.overcommit_memory=1' for this to take effect.
ter-stepanyan-redis     | 1:C 04 Nov 2026 15:11:39.699 * oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
ter-stepanyan-redis     | 1:C 04 Nov 2026 15:11:39.699 * Redis version=7.4.12, bits=64, commit=00000000, modified=0, pid=1, just started
ter-stepanyan-redis     | 1:C 04 Nov 2026 15:11:39.699 # Warning: no config file specified, using the default config. In order to specify a config file use redis-server /path/to/redis.conf
ter-stepanyan-redis     | 1:M 04 Nov 2026 15:11:39.699 * Increased maximum number of open files to 10032 (it was originally set to 1024).
ter-stepanyan-redis     | 1:M 04 Nov 2026 15:11:39.699 * monotonic clock: POSIX clock_gettime
ter-stepanyan-redis     | 1:M 04 Nov 2026 15:11:39.700 * Running mode=standalone, port=6379.
ter-stepanyan-redis     | 1:M 04 Nov 2026 15:11:39.700 * Server initialized
ter-stepanyan-redis     | 1:M 04 Nov 2026 15:11:39.700 * Loading RDB produced by version 7.4.12
ter-stepanyan-redis     | 1:M 04 Nov 2026 15:11:39.700 * RDB age 0 seconds
ter-stepanyan-redis     | 1:M 04 Nov 2026 15:11:39.700 * RDB memory usage when created 0.94 Mb
ter-stepanyan-redis     | 1:M 04 Nov 2026 15:11:39.700 * Done loading RDB, keys loaded: 0, keys expired: 0.
ter-stepanyan-redis     | 1:M 04 Nov 2026 15:11:39.700 * DB loaded from disk: 0.000 seconds
ter-stepanyan-redis     | 1:M 04 Nov 2026 15:11:39.700 * Ready to accept connections tcp
ter-stepanyan-redis     | 1:C 04 Nov 2026 15:05:18.590 # WARNING Memory overcommit must be enabled! Without it, a background save or replication may fail under low memory condition. Being disable
d, it can also cause failures without low memory condition, see https://github.com/jemalloc/jemalloc/issues/1328. To fix this issue add 'vm.overcommit_memory = 1' to /etc/sysctl.conf and then reb
oot or run the command 'sysctl vm.overcommit_memory=1' for this to take effect.
ter-stepanyan-redis     | 1:C 04 Nov 2026 15:05:18.591 * oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
ter-stepanyan-redis     | 1:C 04 Nov 2026 15:05:18.591 * Redis version=7.4.12, bits=64, commit=00000000, modified=0, pid=1, just started
ter-stepanyan-redis     | 1:C 04 Nov 2026 15:05:18.591 # Warning: no config file specified, using the default config. In order to specify a config file use redis-server /path/to/redis.conf
ter-stepanyan-redis     | 1:M 04 Nov 2026 15:05:18.592 * Increased maximum number of open files to 10032 (it was originally set to 1024).
ter-stepanyan-redis     | 1:M 04 Nov 2026 15:05:18.592 * monotonic clock: POSIX clock_gettime
ter-stepanyan-redis     | 1:M 04 Nov 2026 15:05:18.592 * Running mode=standalone, port=6379.
ter-stepanyan-redis     | 1:M 04 Nov 2026 15:05:18.593 * Server initialized
ter-stepanyan-redis     | 1:M 04 Nov 2026 15:05:18.593 * Loading RDB produced by version 7.4.12
ter-stepanyan-redis     | 1:M 04 Nov 2026 15:05:18.593 * RDB age 0 seconds
ter-stepanyan-redis     | 1:M 04 Nov 2026 15:05:18.593 * RDB memory usage when created 0.94 Mb
ter-stepanyan-redis     | 1:M 04 Nov 2026 15:05:18.593 * Done loading RDB, keys loaded: 0, keys expired: 0.
ter-stepanyan-redis     | 1:M 04 Nov 2026 15:05:18.593 * DB loaded from disk: 0.000 seconds
ter-stepanyan-redis     | 1:M 04 Nov 2026 15:05:18.593 * Ready to accept connections tcp
ter-stepanyan-assistant | INFO:     Started server process [1]
ter-stepanyan-assistant | INFO:     Waiting for application startup.
ter-stepanyan-assistant | INFO:     Application startup complete.
ter-stepanyan-assistant | INFO:     Uvicorn running on http://0.0.0.0:8097 (Press CTRL+C to quit)
ter-stepanyan-assistant | INFO:     127.0.0.1:54886 - "GET /health HTTP/1.1" 200 OK
ter-stepanyan-assistant | INFO:     127.0.0.1:46542 - "GET /health HTTP/1.1" 200 OK
ter-stepanyan-assistant | INFO:     127.0.0.1:58218 - "GET /health HTTP/1.1" 200 OK
ter-stepanyan-assistant | INFO:     127.0.0.1:55096 - "GET /health HTTP/1.1" 200 OK
ter-stepanyan-assistant | INFO:     127.0.0.1:43732 - "GET /health HTTP/1.1" 200 OK
```

Рисунок 12 — Логи сервисов

Первые строки лога приложения (рисунок 13) показывают запуск uvicorn на порту 8097. Строки `Waiting for application startup` относятся к подготовке самого приложения FastAPI. Ожидание базы и Redis выполняет Compose ещё до создания процесса приложения, поэтому в логах приложения оно не отображается.

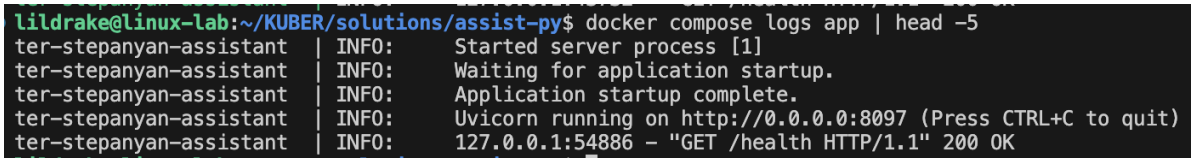


Рисунок 13 — Начало лога приложения

Проверка кэширования показана на рисунке 14. Первый запрос `/weather/Moscow` вернул данные с `"source": "api"`, второй такой же запрос — с `"source": "cache"`. Эндпоинт `/cache/stats` показал владельца `ter-stepanyan` и город `moscow` в кэше. Команда `redis-cli KEYS` нашла ключ `weather:ter-stepanyan:moscow`, а команда `TTL` показала оставшееся время жизни ключа: сначала 11, затем 5 секунд. Ответ `-2` означает, что ключ уже удалён, так как истекло его время жизни (ответ `-1` означал бы ключ без срока жизни).

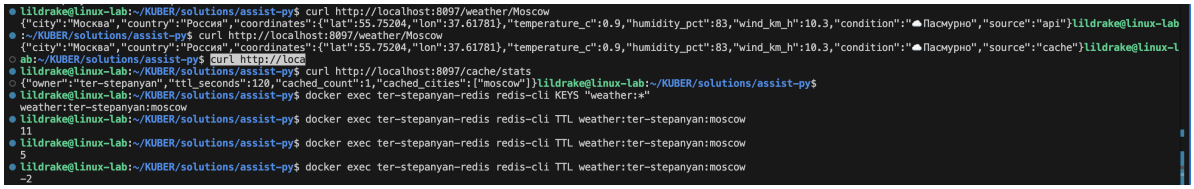


Рисунок 14 — Проверка кэширования

Список сетей с моим префиксом приведён на рисунке 15.

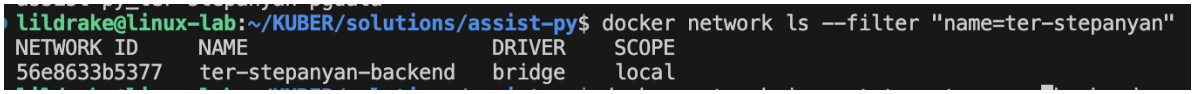


Рисунок 15 — Список сетей

Команда `docker network inspect ter-stepanyan-backend` (рисунок 16) показывает, что все три контейнера подключены к одной сети и получили адреса из её подсети `172.18.0.0/16`.

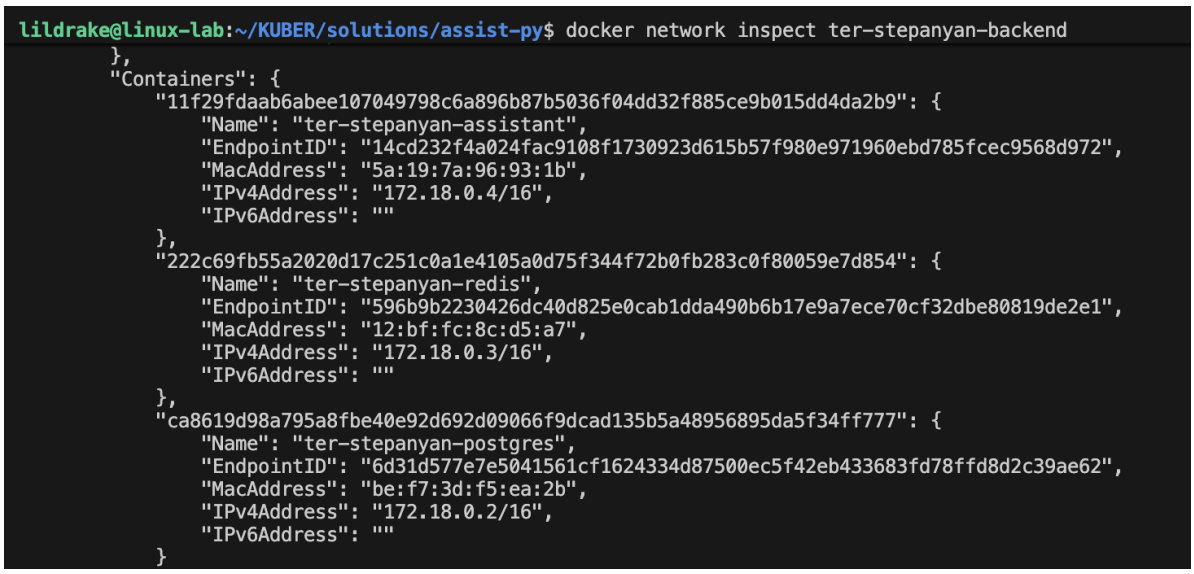


Рисунок 16 — Три контейнера в сети ter-stepanyan-backend

Список томов приведён на рисунке 17: данные PostgreSQL хранятся в именованном томе `ter-stepanyan-pgdata`.

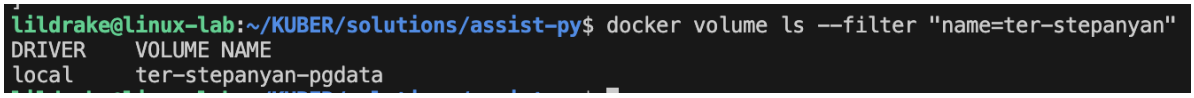


Рисунок 17 — Список томов

1.4 Этап 4. Масштабирование сервиса

Для масштабирования создан файл `docker-compose.scale.yml` (приложение В). У сервиса `app` убран `container_name`, так как имя контейнера должно быть уникальным, а в `ports` указан только порт внутри контейнера `${APP_PORT}`. Порт на хосте Docker назначает каждому экземпляру сам, поэтому три копии не конфликтуют за один порт хоста. Внутри все экземпляры слушают порт 8097, что возможно, потому что у каждого контейнера свой сетевой стек.

PostgreSQL и Redis в этой конфигурации описаны по шаблону задания: порты на хост не публикуются, и доступ к ним есть только у контейнеров внутри сети. Снаружи доступны только экземпляры приложения.

Запуск трёх экземпляров командой `docker compose -f docker-compose.scale.yml up -d --build --scale app=3` показан на рисунке 18. Экземпляры `assist-py-app-1`, `-2` и `-3` получили порты хоста 32777, 32779 и 32778.

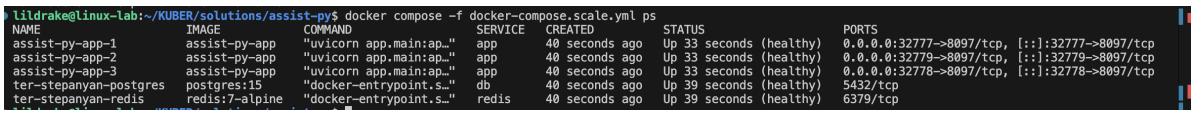


Рисунок 18 — Три экземпляра приложения

Все три экземпляра отвечают на `/health` по своим портам (рисунок 19).

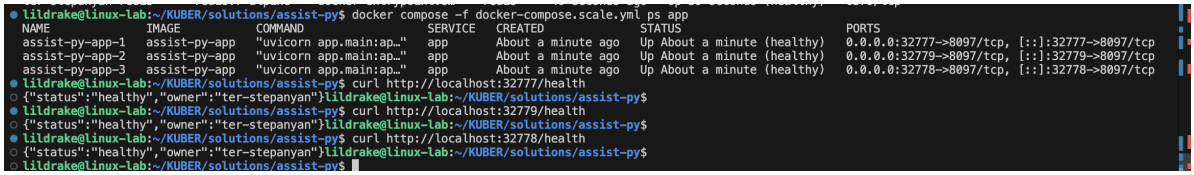


Рисунок 19 — Проверка здоровья экземпляров

Проверка общего кэша показана на рисунке 20. Запрос `/weather/London` к первому экземпляру ушёл во внешний API (`"source": "api"`), а такой же запрос ко второму экземпляру вернул данные из кэша (`"source": "cache"`), хотя это другой контейнер. Третий экземпляр в `/cache/stats` показывает `london` в списке закэшированных городов. Это работает, потому что приложение не хранит состояние внутри себя: заметки находятся в PostgreSQL, а кэш — в Redis, общих для всех экземпляров.

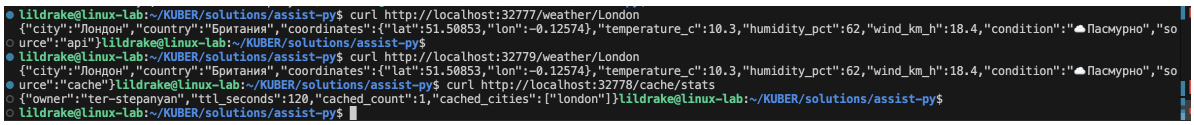


Рисунок 20 — Общий кэш Redis для всех экземпляров

После проверки масштабированная конфигурация остановлена (рисунок 21), и запущена основная (рисунок 22). Compose снова дождался состояния `Healthy` у `redis` и `postgres` и только после этого запустил приложение.

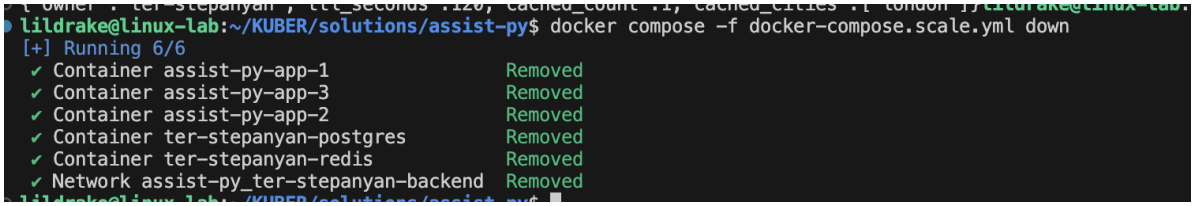


Рисунок 21 — Остановка масштабированной конфигурации



Рисунок 22 — Запуск основной конфигурации

1.5 Этап 5. Финальный артефакт

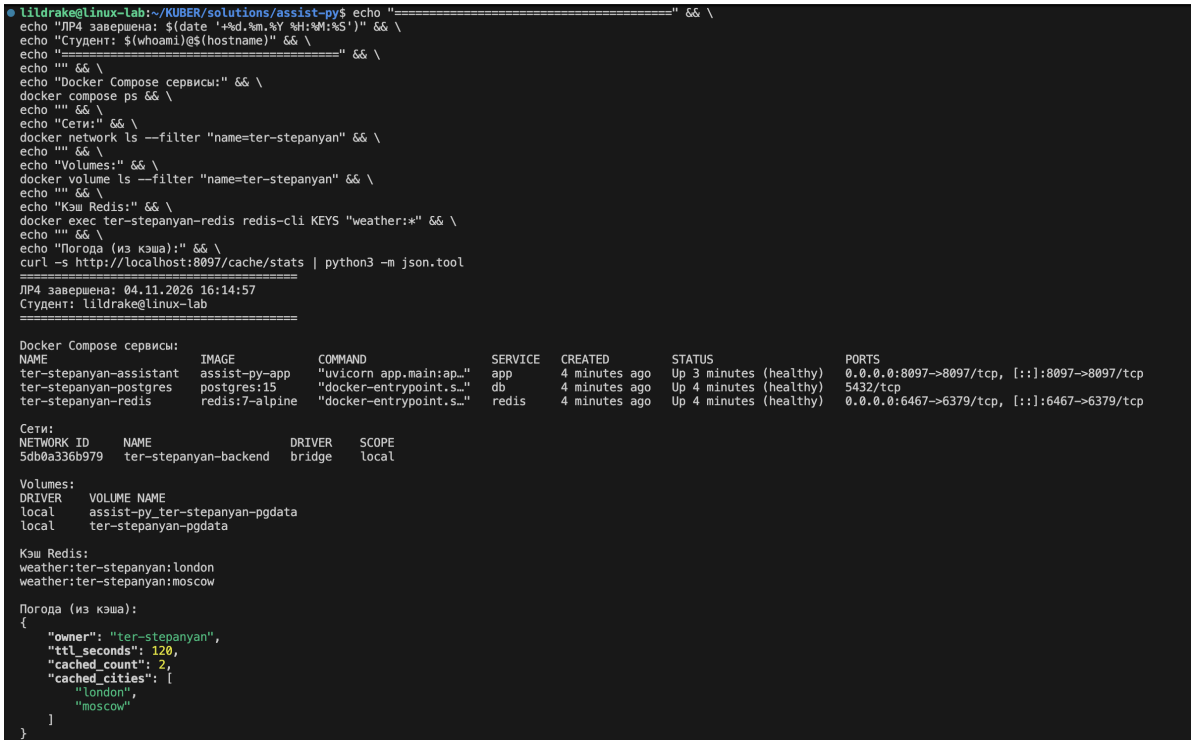


Рисунок 23 — Финальный артефакт

ЗАКЛЮЧЕНИЕ

В ходе работы запуск приложения, PostgreSQL и Redis перенесён в Docker Compose: весь стек описан в одном файле и поднимается одной командой, а персональные параметры вынесены в `.env`. С помощью `healthcheck` и `depends_on` настроен порядок запуска: приложение стартует только после того, как база и Redis готовы принимать подключения.

В приложение добавлено кэширование погоды в Redis с временем жизни 120 секунд: повторный запрос по тому же городу возвращается из кэша без обращения к внешнему API. Проверено, что контейнеры общаются по пользовательской сети по именам сервисов, а проброс портов нужен только для доступа с хоста.

При масштабировании три экземпляра приложения работали одновременно и использовали общий кэш Redis. Это показывает, что приложение можно масштабировать горизонтально, потому что его состояние вынесено в отдельные сервисы.

ПРИЛОЖЕНИЕ А

Содержимое docker-compose.yml

```
services:
  db:
    image: postgres:15
    container_name: ter-stepanyan-postgres
    command: postgres -p ${PG_PORT}
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - ter-stepanyan-pgdata:/var/lib/postgresql/data
    networks:
      - ter-stepanyan-backend
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d
      ↪  ${POSTGRES_DB} -p ${PG_PORT}"]
      interval: 30s
      timeout: 5s
      start_period: 10s
      retries: 3

  app:
    build:
      context: .
      dockerfile: Dockerfile.optimized
    container_name: ter-stepanyan-assistant
    ports:
      - "${APP_PORT}:${APP_PORT}"
    environment:
      APP_OWNER: ${APP_OWNER}
      APP_SECRET: ${APP_SECRET}
      DATABASE_URL: postgresql://${POSTGRES_USER}:${POSTGRES_PASSWOR
      ↪  D}@db:${PG_PORT}/${POSTGRES_DB}
      REDIS_URL: redis://redis:6379/0
      CACHE_TTL: ${CACHE_TTL}
    volumes:
      - ./config:/app/config:ro
    depends_on:
      db:
```

```
38          condition: service_healthy
39        redis:
40          condition: service_healthy
41      networks:
42        - ter-stepanyan-backend
43
44    redis:
45      image: redis:7-alpine
46      container_name: ter-stepanyan-redis
47      ports:
48        - "${REDIS_PORT}:6379"
49      networks:
50        - ter-stepanyan-backend
51      healthcheck:
52        test: ["CMD", "redis-cli", "ping"]
53        interval: 5s
54        timeout: 3s
55        retries: 5
56
57  networks:
58    ter-stepanyan-backend:
59      name: ter-stepanyan-backend
60      driver: bridge
61
62  volumes:
63    ter-stepanyan-pgdata:
64      name: ter-stepanyan-pgdata
```

ПРИЛОЖЕНИЕ Б

Содержимое .env.example

```
APP_OWNER=ter-stepanyan
APP_SECRET=
APP_PORT=8097
POSTGRES_USER=user_97
POSTGRES_PASSWORD=
POSTGRES_DB=assistant_97
PG_PORT=5527
REDIS_PORT=6467
CACHE_TTL=120
```

ПРИЛОЖЕНИЕ В

Содержимое docker-compose.scale.yml

```
services:
  db:
    image: postgres:15
    container_name: ter-stepanyan-postgres
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - ter-stepanyan-pgdata:/var/lib/postgresql/data
    networks:
      - ter-stepanyan-backend
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
      interval: 5s
      timeout: 3s
      retries: 5

  redis:
    image: redis:7-alpine
    container_name: ter-stepanyan-redis
    networks:
      - ter-stepanyan-backend
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5

  app:
    build:
      context: .
      dockerfile: Dockerfile.optimized
    ports:
      - "${APP_PORT}"
    environment:
      APP_OWNER: ${APP_OWNER}
      APP_SECRET: ${APP_SECRET}
```

```
39        DATABASE_URL: postgresql://${POSTGRES_USER}:${POSTGRES_PASSWOR
          ↪ D}@db:5432/${POSTGRES_DB}
40        REDIS_URL: redis://redis:6379/0
41        CACHE_TTL: ${CACHE_TTL}
42      depends_on:
43        db:
44          condition: service_healthy
45        redis:
46          condition: service_healthy
47      networks:
48        - ter-stepanyan-backend
49
50  volumes:
51    ter-stepanyan-pgdata:
52
53  networks:
54    ter-stepanyan-backend:
55      driver: bridge
```